
Project Report

Precision Closed-Loop AC Power Controller using PID

Name: Yojit Kumar

Reg. No.: 20231289

Course Code: PH3144

Group: #23

1 Introduction

In any modern electric system, especially in labs, precise control over the power delivered to an electrical appliance/device is important for efficiency, stability and protection of devices. What we've tried to do is to precisely control the electrical power of any device. We are using a simple light bulb for the ease of demonstration. But in theory, we can implement this on any device connected to AC Mains.

Our device measures the current and voltage input for an electric load and allows us to fix a power in the working range of the appliance. We control this with a technique called *phase-angle fired control*, where the firing angle of a thyristor (Triac) is adjusted to control the portion of the AC Waveform delivered to the load.

For an electric bulb, this works as a dimmer for the system, allowing us to observe the difference. But a simple dimmer is an open-loop control, whilst we have implemented a closed-loop approach with help of *PID*. You cannot ask the dimmer to set the bulb to run at 50 W. You can only set it to 50% effort, but you can't be sure. It is also susceptible to disturbances. If there is an increase in the voltage in the circuit, the dimmer doesn't know that, and the bulb will light brighter.

A closed-loop control is better because it uses the feedback to compare the result. We are constantly measuring the current and voltage to find the actual Power and compare it to the set-point to dynamically adjust the firing angle based on the feedback.

2 Design

2.1 Block Diagram

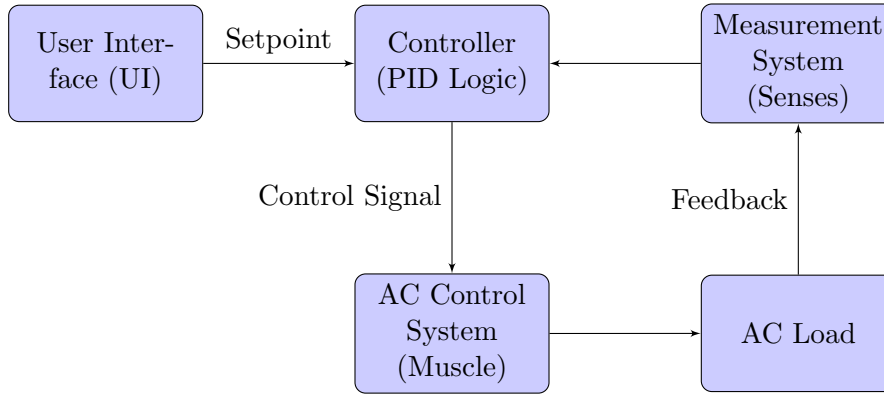


Figure 1: Block Overview of the Closed-Loop Power Controller

We divide our system into smaller task-specific parts to better understand our system. First is the User Interface (UI), it allows us to display the results and also fix a set-point for the system. The Controller (here Arduino) is the brain of the whole system, running the PID logic, and controlling all the other components of our project. The Measurement System consists of some simple sensors that allow us to take current and voltage measurements and send these to the Controller. The AC Control is the muscle of the system, it controls the parameters to maintain the set-point as dictated by the Controller.

2.2 Hardware Design

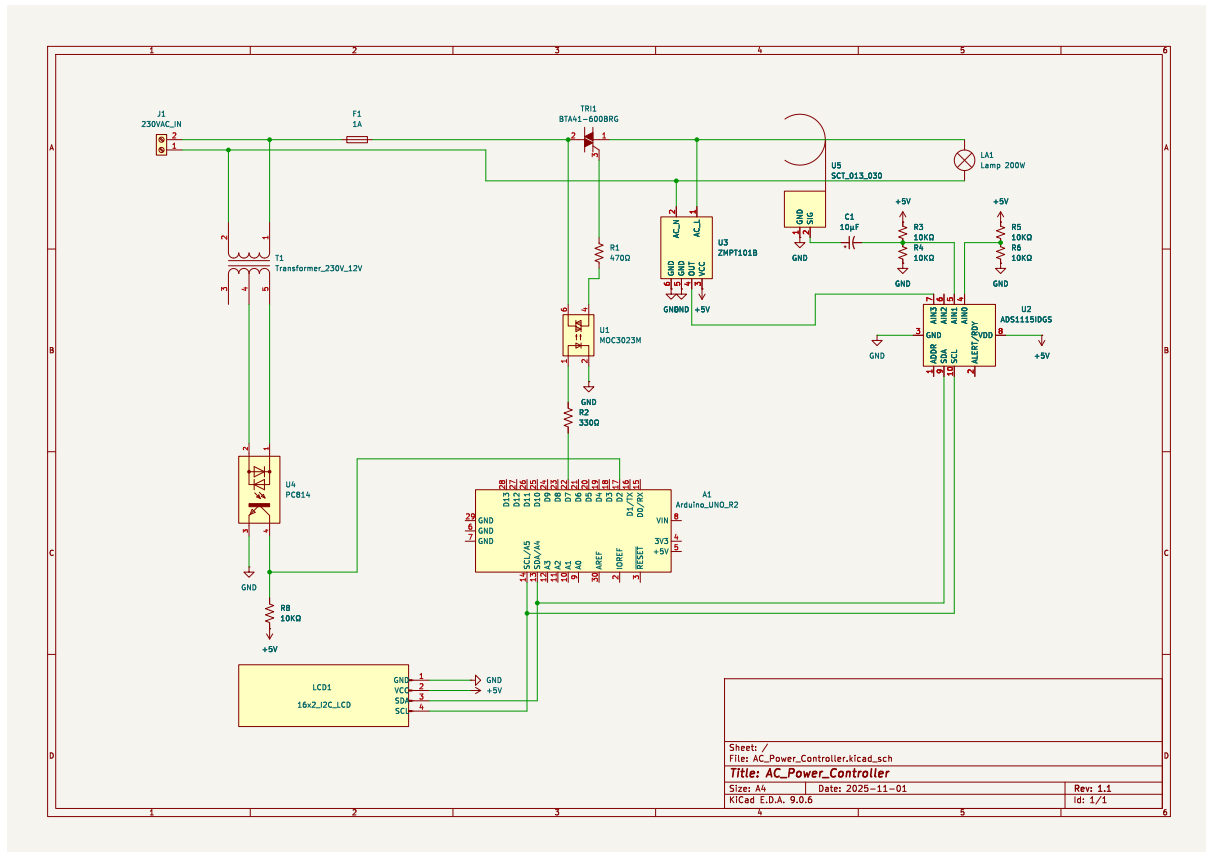


Figure 2: KiCad Schematics for the whole working the circuit

2.2.1 Measurement Module

This module is responsible for providing the critical feedback to the controller. The major components of the measurement module are:

1. ZMPT101B Voltage Sensor Module

This active module safely steps down the 230V AC mains voltage to a low-voltage, isolated AC sine wave. Its onboard op-amp circuit biases this signal around a 2.5volt DC offset, making it safe for the low-voltage components to read.

2. SCT-013-030 Current Sensor

A non-invasive current transformer that clamps around the live wire. It outputs a small AC voltage that is directly proportional to the AC current flowing to the load. This signal is also biased at 2.5V using an external voltage divider.

3. ADS1115 16-bit ADC Module

This is a high-precision, 16-bit Analog-to-Digital Converter. It is connected to the Arduino via the I2C bus (A4/A5). The Arduino's built-in 10-bit ADC is too noisy and has insufficient resolution for a "precision" controller. The 16-bit ADS1115 provides high-accuracy, low-noise measurements.

2.2.2 AC Control Module (Muscle)

This component is the high-voltage part of the circuit that controls the power flow. The major components that make up the AC Control Module are:

1. MOC3023M (Triac Driver Optocoupler)

This chip is the safe bridge between the Arduino and the Triac. It receives a low-voltage 5V "fire" command from the Arduino (on pin D7) and uses an internal light-activated switch to trigger the high-voltage Triac.

This provides galvanic isolation. It is the most important safety component, creating a physical air gap between the 230V AC circuit and the 5V logic circuit, protecting the Arduino and the user from lethal voltage.

2. BTA41600B (Triac)

This is the heavy-duty, high-power AC switch. When it receives the trigger signal from the MOC3023M, it "latches" ON and conducts current to the light bulb. It automatically turns itself OFF (resets) at the end of every AC half-cycle when the current drops to zero.

This is the component that actually controls the power. By precisely timing when this switch is turned on within each 10ms half-cycle, we can control the total energy delivered to the load.

3. PC814 (AC-Input Optocoupler)

This chip functions as our Zero-Crossing Detector (ZCD). Its AC input side watches the sine wave from the step-down transformer. At the precise moment the wave crosses its center point, the optocoupler's internal LED turns off, and triggers the phototransistor on the output side. This interrupt signal pulls down the output PIN_4 that is connected to +5V of the Arduino.

This signal is connected to the Arduino's hardware interrupt pin (D2). It provides a perfect timing pulse 100 times per second, synchronizing the Arduino's code with the AC line.

3 Methodology

In order to control the high-voltage current passing through the load to precisely change the power. We are using *Phase Angle Control* using the **Triac**. The method requires us to change the gate voltage to the triac to control the current through the heavy load.

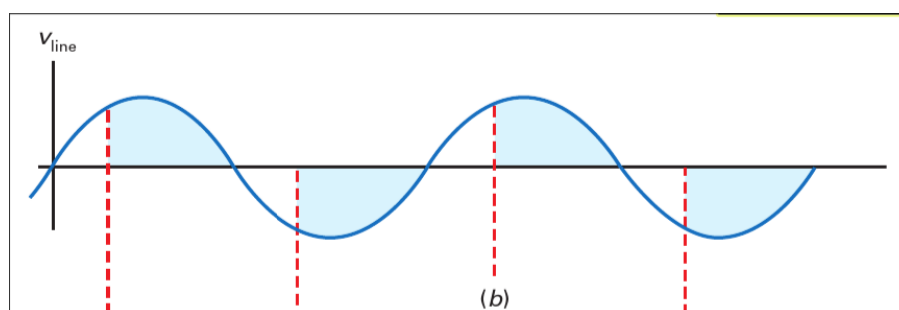


Figure 3: Line voltage for the triac *Ref: Malvino & Bates Pg. 549*

Figure 3 shows the line voltage. When the triac is triggered, the triac starts conducting. It basically means the triac is on, and it will remain on until the line voltage goes back to zero then it turns off again. Figure 4 shows us the voltage across the load during this process. This

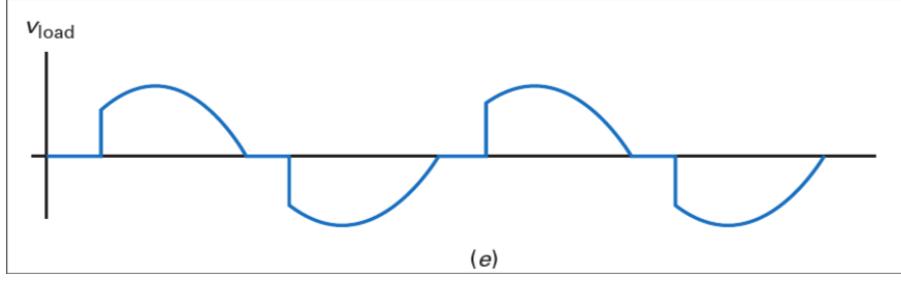
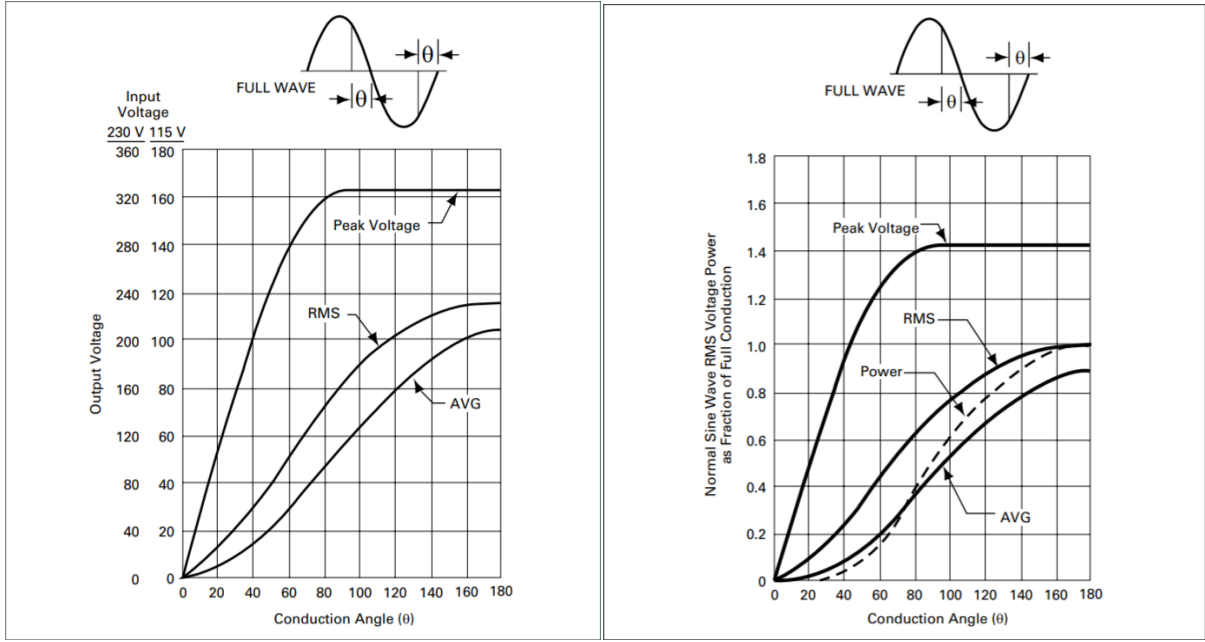


Figure 4: Voltage across a load connected to the triac Ref: *Malvino & Bates Pg. 549*

turning on and off happens for every half-pulse of AC passing through the triac. Thus, the triac is switches its state around 200 times a second.



(a) Voltage of Full-Wave Phase Angle Control (Sinusoidal)

(b) Power of Full-Wave Phase Angle Control

Figure 5: Voltage & Power Curves for different θ Ref: *Littlefuse AN1003 datasheet*

In Figure 5, θ is the conduction angle which is equivalent to $\pi - \theta_{\text{firing}}$. θ_{firing} is proportional to the firing delay. Thus, we can use the curves to study how the power or voltage may vary across the load as we change the firing delay. But, as we are working with a closed-loop feedback system we don't need to refer to the curve, our algorithm is supposed to give us a correction just based on the error from the set point.

In order to find this error term, we need to find the real-time value of the power first. For which we are using a voltage sensing module (*ZMPT101B*) and a current sensing module (*SCT-013-030*), and both of these are connected to an external Analog-to-Digital Converter(ADC) (*ADS1115*) which is there to give us a better resolution than the onboard ADC of the Arduino Board.

Another important part of controlling the AC power and setting a firing delay is to actually know when the AC pulse starts. Because of how a triac functions it will auto turn-off at the end of each half cycle but we need to know the starting point. This starting point is detected using a zero-crossing detector that we have created using a step-down transformer and a AC input optocoupler (*PC814*). We have also attached a basic I2C display to as a part of our user

interface to display the power, voltage and current information in real-time.

4 Implementation

The project was designed with two separate perfboards: high-voltage perfboard as in Figure 6 and a low-voltage perfboard Figure 7a. This was created to keep in mind the safety of low voltage components, and over safe handling of the project. We've also used optocouplers for the same reason; to create circuit isolation.

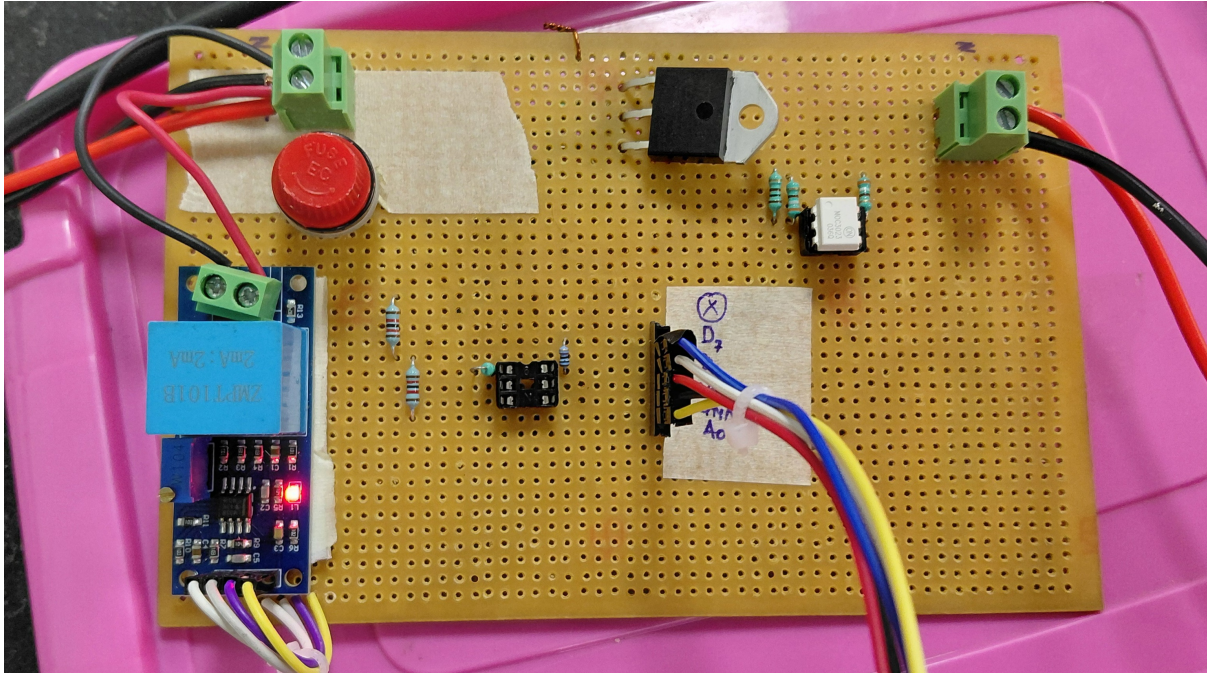


Figure 6: **This is the high-voltage perfboard.** AC-In is from the left side which passes to the triac in the middle and load connected to the rightmost connection point. Triac is connected to the *MOC3023M* IC. The *ZMPT101B* module at the left has a stepdown transformer visible to us as a blue cube. The empty IC base is supposed to hold the *PC814* IC which is the AC input optocoupler, a part of the zero-crossing detector. All connections are soldered at the back and terminate at the jumper base in the middle, which we connect to the low voltage perfboard.

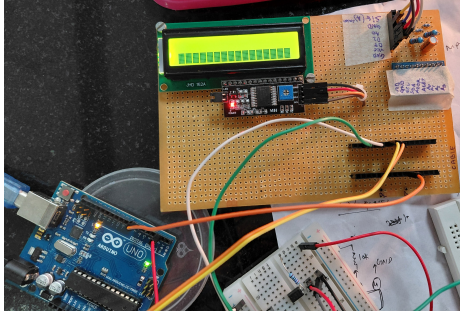
There is an extra component of the project on the breadboard as seen in Figure 7b. It is a zero-crossing detector made using *PC814* IC and a bridge rectifier. The purpose of this circuit is to provide the required interrupt signal as seen in Figure 8. This was created as a fix for a bug in our earlier designs. We will talk more on this in the debugging sections.

5 Software

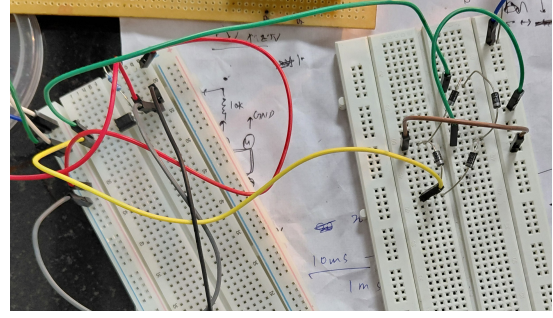
The code that we've written for the project is in C++, it integrates all hardware components into a single, event-driven closed loop system.

We've have used some libraries for the operation of certain sensors, or simply the PID algorithm.

1. `<Wire.h>`: This is for the I2C communication, allowing us to use the *ADS1115* and the LCD Display.
2. `<Adafruit_ADS1X15.h>`: It's a high level interface, specifically for the *ADS1115* IC. It makes the process of setting a gain, and reading 16-bit analog values easier.



(a) This is the low-voltage perfboard. Jumper connections from the first perfboard and the current sensor module join at the top-right. There are voltage dividers and a clamper at the top for the ac-signal from the current-sensor module. The I2C display module is connected on top-left. And all connections terminate at the jumper base at the bottom, that connects to the *Arduino UNO*



(b) This is the zero-crossing detector fix we created. The right breadboard has a bridge-rectifier which is connected to a voltage output of a stepdown 12V transformer. The rectified signal is passing to the *PC814* IC on the bread-board at the right. This simply triggers at the start of each pulse half-cycle and sends a signal to the *Arduino UNO*

Figure 7: Low-voltage Circuit

3. `<PID_v1.h>`: This is our robust, prebuilt PID algorithm, saving us development time.
4. `<Emonlib.h>`: This is to directly communicate with the *SCT-013-030* current sensor module, when using the Arduino's inbuilt ADC.
5. `<ZMPT101B.h>`: This is for direct communication with the voltage sensing module, when using the Arduino's inbuilt ADC.

We initialized the system with the `setup()` function to configure the components before the main loop. `ads.begin()` and the `ads.setGain(GAIN_ONE)` to start the ADS and configures the full 16-bit resolution over a ± 4.096 V range.

Next thing, we initialize is the PID controller. `myPID.SetSampleTime(100)` tells the PID to run every 100 ms and the main PID function is

`PID(&measurePower, &pidOutput, &setpointPower, Kp, Ki, Kd, REVERSE)`

A very important part of the code is the interrupt attachment done through:

`attachInterrupt(digitalPinToInterrupt(ZCD_PIN), onZeroCross, CHANGE)`. It tells the microcontroller to stop whatever it is doing and immediately run the `onZeroCross()`

The main loop is structured to be non-blocking, ensuring that firing logic is never delayed.

```

1 if (zcdFlag == true) {
2   // Wait for the delay calculated by the PID
3   delayMicroseconds(firingDelay);
4
5   // Fire the Triac
6   digitalWrite(TRIAC_PIN, HIGH);
7   delayMicroseconds(20); // Short pulse
8   digitalWrite(TRIAC_PIN, LOW);
9
10  zcdFlag = false; // Reset the flag
11 }
```

This is the most time-sensitive part of the code. It waits for the `firingDelay` (calculated by the PID controller) and then sends a brief $20\mu\text{s}$ pulse to the *MOC3023M*, triggering the Triac for that cycle.

Next we calculate the power using `calculatePower()`. We run this function inside a `millis()` timer. We run a `for` loop that takes many sample readings from the ADC. We

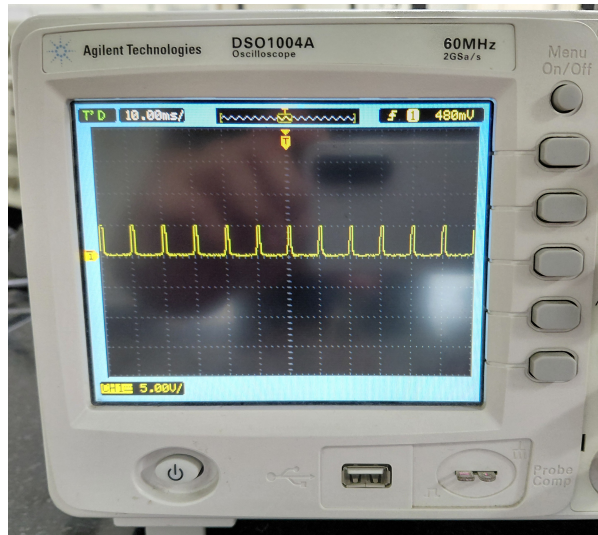


Figure 8: This is the output for the interrupt signal from the zero-crossing detector being fired. We have a pulse/signal every 10 ms as seen on the oscilloscope.

calculate the RMS value of current and voltage using these readings. And finally, we update the global `measuredPower` variable, which the PID controller use.

```

1 // --- FUNCTION TO CALCULATE POWER ---
2 void calculatePower() {
3     // Use 64-bit integers to prevent overflow (the 'nan' bug)
4     unsigned long long sumVoltageSq = 0;
5     unsigned long long sumCurrentSq = 0;
6
7     int16_t voltageSample;
8     int16_t currentSample;
9
10    for (int i = 0; i < sampleCount; i++) {
11        voltageSample = ads.readADC_SingleEnded(VOLT_ADC_S);
12        currentSample = ads.readADC_SingleEnded(CURR_ADC_S);
13
14        long filteredVoltage = voltageSample - voltageOffset;
15        long filteredCurrent = currentSample - currentOffset;
16
17        sumVoltageSq += (unsigned long long)filteredVoltage*filteredVoltage;
18        sumCurrentSq += (unsigned long long)filteredCurrent*filteredCurrent;
19    }
20
21    // Calculate RMS Voltage
22    double meanVoltageSq = (double)sumVoltageSq / sampleCount;
23    double rmsVoltageRaw = sqrt(meanVoltageSq);
24    double Vrms = rmsVoltageRaw * ADS_VOLTS_PER_BIT * VOLTAGE_CAL;
25
26    // Calculate RMS Current
27    double meanCurrentSq = (double)sumCurrentSq / sampleCount;
28    double rmsCurrentRaw = sqrt(meanCurrentSq);
29    double Irms = rmsCurrentRaw * ADS_VOLTS_PER_BIT * CURRENT_CAL;
30
31    // Update the global measuredPower variable
32    measuredPower = Vrms * Irms;
33 }

```

Inside the same 100 ms timer, after the power is calculated, the `myPID.Compute()` function is called. This single command tells the PID library to:

1. Read the `measuredPower` (Input).

2. Compare it to `setpointPower` (Setpoint).
3. Calculate the error.
4. Run its internal P, I, and D logic.
5. Update the `pidOutput` (Output) variable.

Immediately after, the code maps this new output to the `firingDelay`. This mapping is inverted: a low PID output (0, for "min power") results in a long 9500 μs delay, and a high PID output (255, for "max power") results in a short 1000 μs delay. This is what steers the system toward the setpoint.

6 Debugging

6.1 AC Control Failure

During initial testing, the load (light bulb) failed to energize, even when the microcontroller sent a trigger signal.

- **Diagnostics:** A "Dumb Blinker" test sketch was developed to bypass the zero-crossing detector and trigger the triac blindly at a fixed interval. This isolated the fault to the *MOC3023M*/Triac circuit. We verified the load by connecting it directly to mains (Passed). We probed the *MOC3023M* input (Pin 1/2) with a multimeter. The Arduino was correctly delivering 1.2V during the "ON" state (Passed). Finally, we probed the *MOC3023M* output (Pin 4/6). No conduction was detected despite the input signal. (Failed)
- **Cause:** The *MOC3023M* optocoupler output stage had failed open. Further analysis of the initial soldering revealed that the gate current-limiting resistor (470 Ω) was initially placed incorrectly, likely causing a massive current surge through the phototriac during the first power-up event.
- **Solution:** The *MOC3023M* was replaced. The wiring was corrected to ensure the 470 Ω resistor was placed strictly in series between the MOC output pin and the triac Gate to limit surge current to safe levels.

6.2 Zero-Crossing Detection Failure

The synchronization circuit proved to be the most complex subsystem to debug, requiring multiple iterations and component changes. Initially, the ZCD interrupt service routine never triggered, despite correct wiring. A manual examination of the IC revealed that the *H11AA1* IC was dead, and due to unavailability of the same chip we obtained a *PC814* IC as replacement. Even with replacement the system failed to achieve phase control. The interrupt service routine was either not firing or firing erratically only during power-on transients.

- **Diagnostics:** An oscilloscope was used to visualize the analog output of the *ZMPT101B* voltage sensor. We also used a function generator to replicate the signal from the voltage sensor module and then alter it to figure out the point of failure. We needed to overcome the forward voltage ($V_f \approx 1.2\text{ V}$) of the optocoupler's internal LEDs.
- **Cause:** The signal was found to be biased at 2.5V DC. The *PC814* forward voltage was almost always getting exceeded by the signal because the signal oscillated between 1.5V and 4V, it was always higher than 1.2V. This meant the *PC814* was permanently forward-biased (ON), holding the interrupt pin LOW.

- **Solution** We replaced the *ZMPT101B* module on the input side and replaced it with a step-down transformer of 230V to 12V kind. We connected the the center tap of the output side to the *PC814* IC and provided a 6V peak signal. We also added multiple high wattage resistors before the IC so the signal not exceed the maximum forward current rating.

6.3 Measurement & ADC Integration

RMS calculations for voltage and current were inaccurate or returning `nan` (Not a Number) errors.

- **Diagnostics:** After running for several seconds, the RMS value would become corrupted.
- **Cause:** The `sumVoltageSq` variable, used to accumulate squared ADC samples, was declared as a `long` (32-bit). With 500 samples of 16-bit squared data, the value exceeded the maximum limit (2.1×10^9), causing an integer overflow to a negative value. Attempting to calculate the `sqrt()` of this negative number resulted in `nan`.
- **Solution:** Accumulator variables were promoted to `unsigned long long` (64-bit) to prevent overflow.

7 Results & Discussion

While we aimed to implement a fully integrated, real-time PID control loop, the final execution was constrained by the hardware resources of the microcontroller. However, extensive unit testing successfully validated the electronic design, the measurement accuracy, and the phase-control logic.

Before attempting full integration, individual modules were subjected to rigorous testing to verify their reliability. To verify the "Muscle" (*MOC3023M* and *BTA416*) and "Timing" (*PC814* ZCD) circuits, a hard-coded delay test was performed. The system successfully triggered the Triac at precise intervals (e.g., 5000 μ s after zero-crossing). The load (light bulb) maintained a stable, flicker-free brightness consistent with the firing angle. This confirmed that the ZCD interrupt was firing correctly at 100Hz and the isolation circuit was functioning safely. A "Looper" test code was implemented to sweep the firing delay from 1000 μ s (Max Power) to 9000 μ s (Min Power) and back. The bulb dimmed and brightened smoothly without stuttering or erratic flashing. This proved that the system could handle dynamic changes in firing angle in real-time, a prerequisite for PID control. The sensing module was tested using the RMS calculation algorithm. The system successfully calculated `Vrms`, `Irms`, and `Apparent Power`. The readings were compared against standard multimeter values. The 16-bit *ADS1115* provided sufficient resolution to detect small changes in load current, validating the "Feedback" portion of the control loop.

The primary challenge encountered during the final phase was the integration of all software modules onto the Arduino Uno (ATmega328P). The Arduino Uno possesses only 2KB of Dynamic Memory (SRAM). The project required several memory-intensive libraries to run simultaneously. We understand that there might be some clever or more efficient ways to resolve this issue, but due to our groups inexperience in *C++* proved to be a big obstacle in this. We had to chose to restric ourselves to only demonstrate the individual components of our project.

The original plan included testing the system with a Variac. By artificially lowering the input voltage (simulating a "brownout"), we intended to prove that the PID loop would automatically decrease the firing delay to maintain constant power at the bulb. Since the full PID loop could not run stably alongside the measurement code, this dynamic compensation could not be empirically verified. This validation test was not conducted at the end due to project timeline and limited resources.

Despite the integration setback, we believe that we have successfully delivered a professional-grade hardware platform. The hardware design, safety isolation features, and sensor calibration methods were proven effective. The electronic hardware is fully capable of supporting closed-loop control; the limitation lies strictly within the computational capacity of the chosen microcontroller.